

INFO4990 Honours Thesis Draft

Skill Representation for Scalable Agent Skill Retrieval under Semantic Confusability

Jacky Zhang

The University of Sydney

Draft date: June 6, 2026

Abstract

Large language model agents increasingly rely on reusable skills: procedural capability units that may package instructions, workflow steps, scripts, examples, resources, dependencies, and constraints. As skill libraries scale, the central difficulty is not only storing many capabilities, but selecting the right capability when multiple skills appear semantically similar. A request about extracting information from a PDF, for example, may plausibly match skills for layout review, summarisation, citation grounding, field extraction, OCR, or format conversion, even though these skills require different procedures.

This thesis investigates what information agent skill representations must preserve to distinguish semantically similar but procedurally distinct skills at scale. It separates three layers that are often conflated: the stored skill artifact, the selection representation exposed to a retriever, and the retrieval policy that produces a candidate set before the main agent loads full skill documentation. The thesis evaluates flat metadata, full skill artifacts, extracted procedural fields, dependency/resource-aware fields, dense retrieval, generic neural reranking, SkillRouter-style skill-specific retrieval, deterministic procedural reranking, and planned tree/graph structure-aware retrieval.

The current benchmark contains 2401 skills, including 137 controlled evaluated skills, 1800 generated background-scale skills, 460 imported public skills, and 4 support skills. It includes 137 controlled prompts plus a separate 32-prompt public-gold validation stratum. Validation checks cover integrity, procedural alignment, semantic confusability, prompt leakage, scale pressure, public-skill field prevalence, and gold-label stability. Current results show that flat metadata and generic full-skill embedding alone remain fragile under scale. SkillRouter’s released skill-specific embedding and reranker provide a strong baseline, reaching 83.2% top-1 on controlled prompts. Combining SkillRouter embedding with the thesis field-aware reranker reaches 86.9% top-1 and 0.927 MRR on the controlled benchmark, but does not improve public-gold top-1. This split result suggests that procedural fields are useful when extracted and matched reliably, while public-authored skills still require better normalization and semantic field matching.

The expected contribution is therefore not a claim that one provider or retriever is universally best. Instead, the thesis contributes a framework for analysing skill retrieval as an information-preservation problem, a benchmark for procedurally confusable skill retrieval, and empirical evidence about which fields and retrieval architectures help agents select the right procedural capability under scale.

Declaration and Acknowledgements

Placeholder. Add the final University-required academic integrity, authorship, ethics, and AI-use declaration here if your unit or supervisor provides one. The public USYD unit pages emphasize research ethics and copyright awareness, but the exact declaration wording should come from Canvas, the school, or your supervisor.

Placeholder. Optional acknowledgements section. Keep this short unless your school requires a specific format.

Contents

Abstract	i
Declaration and Acknowledgements	ii
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Research Questions	2
1.4 Scope	2
1.5 Contributions	3
1.6 Thesis Structure	3
2 Literature Review	4
Version Note	4
2.1 Introduction	4
2.2 Conceptual Dimensions of Skill Retrieval	5
2.3 Why Skills Are a Distinct Retrieval Problem	6
2.4 Evidence That Skill Scaling Is a Real Problem	7
2.5 Flat and Weakly Structured Skill Representations	8
2.6 Structure-Aware Skill Representations	8
2.7 Hierarchical Skill Retrieval	8
2.8 Graph-Based and Relational Skill Organization	9
2.9 Transferable Ideas from Tool and Document Retrieval	10

2.10	Recent Skill Retrieval Benchmarks and Routers	10
2.11	Evaluation Practices and Unresolved Gaps	11
2.12	Synthesis and Current Research Questions	12
2.13	Update Notes for Thesis Revision	12
3	Conceptual Framework	13
3.1	Skill Artifacts, Representations, and Retrieval Policies	13
3.2	The Core Failure Mode	13
3.3	Representation Conditions	14
3.4	Retrieval Architecture Families	14
3.5	Tree and Graph Structure	15
3.6	Working Hypotheses	16
4	Benchmark Design	17
4.1	Design Goals	17
4.2	Skill Library Composition	17
4.3	Controlled Confusable Clusters	18
4.4	Background and Public Skills	18
4.5	Public-Gold Validation Stratum	19
4.6	Gold Labels and Acceptable Alternatives	19
4.7	Validation Rubric	20
4.8	Current Validation Status	21
5	Methodology	22
5.1	Experimental Overview	22
5.2	Representation Conditions	23
5.3	Retrieval and Reranking Methods	24
5.4	M6-v0 Diagnostic Reranker	25
5.5	M6-v1 Field-Aware Procedural Reranking	25
5.6	SkillRouter Hybrid Experiment	26

5.7	M4 Tree Routing	27
5.8	M5 Graph Retrieval	27
5.9	Evaluation Metrics	29
5.10	Downstream Validation Plan	29
5.11	Reproducibility	30
6	Results	31
6.1	Benchmark Validation Results	31
6.2	Local Selector Results	31
6.3	Field Ablation Results	32
6.4	Representation-by-Architecture Coverage	32
6.5	Public-Gold Horizontal Representation Results	33
6.6	Provider and SkillRouter Results	34
6.7	M6-v1 Field-Aware Reranker Results	34
6.8	SkillRouter plus M6-v1 Hybrid Results	35
6.9	Public-Gold Results	35
6.10	M0 Progressive Disclosure	36
6.11	Low-Information and Implicit-Field Cases	37
6.12	Interim Interpretation	37
7	Discussion	38
7.1	What the Results Suggest about Representation	38
7.2	Why M6-v0 Is Diagnostic, Not Final	38
7.3	What the SkillRouter Hybrid Adds	39
7.4	Information That Appears Important	39
7.5	Tree and Graph Methods	40
7.6	Benchmark Validity	40
7.7	Failure Modes	41
7.8	Implications for Agent Skill Systems	41
7.9	Limitations	42

8 Conclusion	43
A Benchmark Rubric	47
A.1 Validation Steps	47
A.2 Current Pass/Warning Summary	48
B Prompt and Skill Examples	49
B.1 PDF Table Extraction	49
B.2 GitHub CI Root Cause	49
B.3 Observability Trace Diagnosis	50
B.4 Public-Gold Example: Figma Implementation	50
B.5 Public-Gold Ambiguity Example	50

Chapter 1

Introduction

1.1 Background

Large language model (LLM) agents are increasingly designed as systems that combine reasoning, action, memory, and external capability use rather than as standalone text generators. Early work on retrieval-augmented generation showed that language models can improve performance by accessing external information at inference time rather than relying solely on knowledge stored in model parameters [1, 2]. Later work on tool use extended this idea from document retrieval to action execution by enabling models to call external functions and APIs [3, 4, 5].

More recently, agent systems have begun to treat reusable capabilities as *skills*. A skill is not simply an atomic tool call. It may include instructions, workflow steps, scripts, examples, dependencies, input requirements, output expectations, and constraints. In practical systems such as filesystem-based agent skill libraries, the full skill artifact may be stored as a directory containing a `SKILL.md` file plus optional resources. The agent may initially see only a compressed representation, such as a name and description, before deciding whether to load the full artifact.

This shift changes the retrieval problem. If the object being selected is a high-level procedure rather than an atomic function, retrieval quality depends not only on topical relevance, but also on procedural suitability. Two skills may look semantically similar because they refer to the same domain or task family, while still differing in preconditions, workflow steps, required resources, output format, or side effects.

1.2 Problem Statement

The problem addressed in this thesis is semantic confusion in large agent skill libraries. As skill libraries grow, a user request may match many plausible skills at the level of names and descriptions. The correct skill may exist, but the retrieval system may surface a nearby skill that sounds relevant while being procedurally wrong.

This failure mode is different from ordinary retrieval failure. The wrong result is often not unrelated. It is close enough to be tempting. For example, a request to “extract information from this PDF” may plausibly match document review, document summarisation, citation grounding, field extraction, or layout-preserving conversion. These skills may all mention documents, extraction, or PDFs, but they require different procedures and produce different outputs. A flat description may not preserve the procedural distinctions required for correct selection.

The scalability issue makes this worse. In a small library, the main agent may be able to inspect all skill names and short descriptions in context. In a larger library, even this metadata can become too large or noisy. The system then needs a candidate-subsetting stage before the main agent loads any full skill document. This makes the selection representation critical: the correct skill must be surfaced before the full artifact can be inspected.

1.3 Research Questions

This thesis uses the following working research questions:

RQ1: What information must agent skill representations preserve to distinguish semantically similar but procedurally distinct skills at scale?

RQ2: How do retrieval strategies that use flat text, dense embeddings, structured procedural fields, tree or graph structure, or hybrid reranking differ in accuracy, efficiency, and failure modes when applied to scalable skill libraries?

These questions revise an earlier framing that asked more generally how structure-aware representations improve over flat description-based retrieval. The revised framing keeps retrieval strategies in scope, but makes representation quality the central research object. The thesis therefore asks not only which method performs best, but what kinds of information are being preserved, lost, or exploited by each method.

1.4 Scope

The thesis focuses on retrieval and candidate selection before full skill execution. It does not attempt to build a complete general-purpose agent operating system. The main agent, the retriever, and the reranker are treated as separable components. The retriever may be a lexical method, an embedding model, a deterministic schema scorer, a neural reranker, or a hybrid pipeline. What matters is whether the retrieval stage can produce a candidate set that contains and ranks the procedurally correct skill.

The evaluation focuses on skill libraries where some skills are intentionally confusable. This is important because broad relevance retrieval is too easy for the research question. A benchmark that only asks whether a document task retrieves a document skill would not test the central failure mode. The difficult case is selecting between skills that share the same broad domain but

differ in operational requirements.

1.5 Contributions

This thesis aims to make four contributions:

1. A conceptual framework that separates skill artifacts, selection representations, and retrieval policies.
2. A benchmark design for scalable skill retrieval under semantic confusability and procedural distinctness.
3. An empirical comparison of flat metadata, full skill artifacts, structured procedural representations, hybrid retrieval-reranking methods, and planned tree/graph structure-aware architectures.
4. A failure-mode analysis identifying what information is needed for robust skill selection, including inputs, outputs, preconditions, workflow constraints, dependencies, resources, negative boundaries, and meta-level task roles.

1.6 Thesis Structure

Chapter 2 reviews the literature on retrieval-augmented language models, tool use, agent skills, skill-library scaling, and structure-aware retrieval. Chapter 3 defines the conceptual framework used throughout the thesis. Chapter 4 describes the benchmark design. Chapter 5 explains the methodology and evaluation metrics. Chapter 6 reports current and planned results. Chapter 7 discusses implications and failure modes. Chapter 8 concludes the thesis and outlines future work.

Chapter 2

Literature Review

Version Note

This chapter preserves the Assignment 3 literature review as the base thesis literature review, then updates the synthesis toward the current thesis framing. The newest 2026 skill-retrieval papers should be rechecked before final submission, because several are recent preprints rather than settled archival publications.

2.1 Introduction

Large language model (LLM) agents are increasingly designed as systems that combine reasoning, action, memory, and external capability use rather than as standalone text generators. Early work on retrieval-augmented generation showed that language models can improve performance by accessing external non-parametric memory at inference time rather than relying solely on knowledge stored in model parameters [1, 2]. Later work on tool use extended this idea from document retrieval to action execution by enabling models to call external functions and APIs [3, 4, 5]. More recently, the literature has begun to treat agent skills as a distinct abstraction: reusable procedural capabilities that may package instructions, scripts, workflows, tools, and task-specific resources into units that can be invoked across tasks [6, 7, 8]. Unlike atomic tools, which typically expose a specific callable function, skills operate as higher-level procedural modules. This shift is important because it changes the retrieval problem. If the objects being selected are not atomic tools but higher-level procedures, then retrieval quality depends not only on semantic relevance but also on procedural suitability.

The research problem addressed in this review is therefore not simply whether large capability libraries are difficult for agents to manage. That broad point is already well established. The more specific issue is whether current methods can distinguish between skills that appear semantically similar at the level of names or descriptions, yet differ in preconditions, workflow steps, dependencies, outputs, or side effects. This distinction matters particularly in personal-

agent settings, where users may accumulate many overlapping skills over time without maintaining a carefully curated capability library. In such settings, flat skill descriptions may create ambiguity even when the correct skill exists in the library. A retrieval system may therefore fail not because a capability is missing, but because the representation and selection process do not preserve procedural differences strongly enough.

This literature review is organised around that problem. Rather than summarising papers one by one, it examines the literature through the questions most relevant to scalable skill retrieval. First, why should skills be treated as a distinct retrieval problem rather than as a simple extension of tool selection? Second, what does the emerging skill literature reveal about the scaling problem itself? Third, what representations and retrieval mechanisms have been proposed for skills, and what limitations remain? Finally, what evaluation practices are currently used, and do they adequately test semantic confusability, context efficiency, and downstream task performance? The review therefore compares prior work not only in terms of whether it improves overall agent behaviour, but also in terms of what kind of skill representation it assumes, how retrieval is performed, and how convincingly it is evaluated under scale. The review argues that the literature already provides strong evidence that flat or weakly structured skill selection becomes unreliable as skill libraries grow, and that structure-aware approaches are promising. However, it also argues that existing work does not yet provide a clear cross-family comparison of which representational properties most effectively distinguish semantically similar but procedurally different skills while remaining retrieval-efficient. This gap motivates the research questions stated at the end of the review.

2.2 Conceptual Dimensions of Skill Retrieval

A useful conceptual distinction is needed before comparing the literature. A *skill artifact* is the actual stored capability, such as a prompt module, executable program, or folder of instructions and scripts. A *selection representation* is the form in which that skill is made available for retrieval or routing, such as a short textual description, a structured schema, a hierarchy position, or a graph node. A *retrieval policy* is the mechanism that selects candidates, such as direct prompt exposure, dense retrieval, hierarchical traversal, graph exploration, or hybrid reranking. Finally, an *evaluation protocol* specifies what success means: retrieval accuracy, downstream task success, context efficiency, robustness under scale, or some combination of these dimensions. This distinction matters because many systems are discussed as if artifact, representation, and retrieval policy were the same thing. In practice, however, a skill may be stored as code or documentation while being represented for selection through only a short textual descriptor. Similarly, a system may use a hierarchical index without changing the internal structure of each skill. Making these layers explicit allows a clearer comparison of what prior work actually contributes.

2.3 Why Skills Are a Distinct Retrieval Problem

The first important question is whether skills are genuinely different from tools for the purposes of retrieval. The recent skill literature strongly suggests that they are. The survey *Agent Skills from the Perspective of Procedural Memory* conceptualizes skills as reusable procedural memories that enable an agent to reuse problem-solving procedures rather than repeatedly reasoning from scratch [6]. This framing is useful because it highlights the procedural nature of skills. The survey organizes the field into acquisition, representation, invocation, and refinement, showing that skills can be encoded as prompts, programs, APIs, or more complex procedural modules. Its main strength is to clarify that a skill is not just a callable function, but a reusable unit of procedure. However, the paper is primarily taxonomical. It does not provide a benchmark or retrieval method that shows how agents should select among many semantically overlapping skills. Its importance for the present thesis lies in problem framing rather than direct methodological evidence.

A similar point is developed more strongly in the *SoK: Agentic Skills — Beyond Tool Use in LLM Agents* [7]. That paper argues that skills extend beyond tools, plans, or memory because they operate as executable procedural modules that can encapsulate complex task knowledge. This is directly relevant to the present topic because it implies that retrieval methods designed for atomic tools may not transfer cleanly to skills. If tools are primarily selected on the basis of function-level relevance, while skills must be selected on the basis of procedural appropriateness, then the retrieval target itself changes. The value of the SoK paper is therefore conceptual: it strengthens the argument that skill retrieval should not be treated as merely a larger version of tool retrieval. However, like the procedural-memory survey, it does not yet solve the practical problem of large-scale skill selection.

Early skill systems further show why this distinction matters. SkillAct introduces textual skill abstractions represented as a tuple consisting of a name, natural-language instructions, and example trajectories, all appended directly to the prompt of a baseline agent [9]. The paper demonstrates that skill abstractions can improve performance, especially by reducing execution errors in ALFWorld. Its contribution is important because it provides an early and transparent baseline for skill-augmented agents. However, its retrieval model is effectively absent: the available skills are exposed as a flat prompt list. This means the system works only because the experimental setting is small enough for full prompt inclusion to remain manageable. The paper therefore supports the claim that skills are useful, but also exposes the weakness of flat prompt-level skill access when libraries become large or internally redundant.

Voyager provides a stronger architectural example of a skill library [10]. It stores successful behaviors as executable code and indexes them by embeddings of their natural-language descriptions. This is significant because it shows that skills can function as reusable external capabilities rather than one-off generated plans. The paper demonstrates that a growing skill library improves exploration and transfer in Minecraft. However, its retrieval mechanism remains shallow: skills are recalled through embedding similarity over descriptions rather than through procedural structure. As a result, Voyager is best understood as a strong early precursor to skill-library

architectures rather than a complete answer to scalable skill retrieval. It demonstrates the utility of externalized skills, but does not study what happens when multiple skills share similar language while differing in operational logic.

2.4 Evidence That Skill Scaling Is a Real Problem

The next major theme in the literature concerns whether the scaling problem is real and, if so, what exactly causes it. Three recent papers are especially important here: *SkillsBench*, *When Single-Agent with Skills Replace Multi-Agent Systems and When They Fail*, and *Agent Skills: A Data-Driven Analysis of Claude Skills* [11, 12, 13]. Together, these papers shift the discussion away from generic claims about capability growth and toward more specific evidence about skill-library behavior under scale.

SkillsBench evaluates whether skills actually help LLM agents across diverse tasks by comparing conditions with no skills, curated skills, and self-generated skills [11]. Its most useful finding is not merely that curated skills improve performance, but that smaller and more focused skill sets often outperform larger and more comprehensive ones. This matters because it suggests that the effectiveness of a skill library depends not only on the presence of reusable procedures, but also on how tightly they are scoped and represented. The benchmark is valuable because it tests skill usefulness across domains rather than assuming that more skills are always better. However, it only weakly addresses the retrieval problem. The relevant curated skills are effectively assumed to be available, so the study does not directly evaluate how the correct skill should be retrieved from a large ambiguous library. Its contribution is therefore diagnostic rather than decisive: it shows that skill packaging matters, but not how scalable skill selection should be performed.

The paper *When Single-Agent with Skills Replace Multi-Agent Systems and When They Fail* is even more directly relevant [12]. It formalizes skills as descriptor-policy-backend tuples and studies how skill selection behaves as the number of available skills increases. Its most important contribution is the finding that performance remains relatively stable until a threshold is reached, after which selection accuracy drops sharply. Crucially, the paper shows that this degradation is driven not merely by the number of skills, but by semantic confusability among them. This is one of the strongest direct pieces of evidence for the present thesis. It isolates semantic overlap as the core failure mode of flat skill selection. At the same time, the paper is weaker as a solution paper. Its proposed hierarchical mitigation is intentionally simple and does not determine which hierarchy design or retrieval structure is best. Nor does it represent skills through rich procedural structure. The paper therefore motivates the research problem very strongly, but does not yet solve it.

The data-driven analysis of Claude skills contributes a different kind of evidence [13]. Rather than proposing a new retrieval method, it measures the properties of a large public skill ecosystem, including redundancy, length variation, and supply-demand imbalance. This is highly relevant because it shows that real skill libraries are already noisy, redundant, and unevenly structured. The paper is particularly useful for context-efficiency arguments, since it shows that while many

skills are short, a non-trivial minority are long enough to create prompt-budget pressure if loaded indiscriminately. It also highlights that discovery often happens through lightweight metadata before full instructions are loaded. However, the paper is not a retrieval-comparison study. Its contribution is ecosystem diagnosis rather than direct evidence about which retrieval methods work best. Still, it supports the claim that scalable discovery and selective loading are practical necessities rather than theoretical luxuries.

2.5 Flat and Weakly Structured Skill Representations

Having established that skills are distinct from tools and that large skill libraries create real confusability problems, the next question is how existing work represents skills for selection. The simplest family of representations remains flat or weakly structured textual skills, as seen in SkillAct and parts of Voyager [9, 10]. These approaches work because they make reusable procedures accessible to the model with minimal infrastructure. Their strengths are simplicity, interpretability, and ease of authoring. Their weakness is that the differentiating information remains embedded in free-form text. A flat description can indicate what a skill is roughly for, but it may not clearly encode conditions under which one apparently similar skill should be preferred over another. This limitation becomes especially serious when the representation available to the retriever is shorter and shallower than the full skill artifact. Flat prompt-based skill systems therefore make a valuable baseline, but not a scalable endpoint.

2.6 Structure-Aware Skill Representations

A more promising direction is to introduce structure explicitly into the skill representation itself. One example is *Learning Agent Skills from Demonstrations and Generating Knowledge Graph*, which decomposes complex demonstrations into meta-skills, generates behavior trees, and transforms the resulting information into a knowledge graph [14]. This paper is relevant because it shows that skills can be represented through explicit internal structure rather than only through free-form descriptions. By moving from trajectories to behavior trees and graph representations, it makes the procedural nature of skills more visible and interpretable. However, the paper is heavily tailored to robotics assembly domains and relies on a predefined meta-skill vocabulary. Moreover, the graph serves more as an organizational and visualization layer than as a thoroughly evaluated large-scale retrieval mechanism. Its main value for the present review is therefore to support the general idea that skills can be decomposed into richer procedural structures.

2.7 Hierarchical Skill Retrieval

Two more directly relevant large-scale skill infrastructure papers are AgentSkillOS and SkillNet [15, 16]. AgentSkillOS organizes skills into a hierarchical capability tree and then uses this

structure for retrieval and DAG-based orchestration. Its strongest contribution is empirical: it demonstrates that structured retrieval can approximate oracle skill selection and that structured orchestration improves downstream task performance compared with flat invocation. This makes it one of the most important system-level papers for the present thesis. However, the capability tree assumption is also its main limitation. Real skills often belong to multiple overlapping functional categories, and a strict tree risks brittle routing decisions. Moreover, the evaluation focuses on overall system performance and orchestration quality rather than isolating retrieval accuracy under highly confusable skill competitors. AgentSkillOS therefore provides strong evidence that structure helps, but weaker evidence about which structural properties are specifically responsible for disambiguating semantically similar skills.

2.8 Graph-Based and Relational Skill Organization

SkillNet takes a broader relational approach [16]. It introduces a taxonomy layer, a relation graph with typed edges such as similarity, composition, and dependency, and a package layer for deployment. This paper is particularly relevant because it refuses to treat skills as isolated items and instead models them as connected assets within a structured network. This is conceptually closer to the needs of large skill ecosystems, where similarity and compositionality often coexist. The main limitation is that the paper functions more as a full infrastructure proposal than as a clean retrieval-comparison study. It provides evidence that structured skill libraries improve downstream performance, but offers less fine-grained analysis of retrieval accuracy, context cost, or robustness under semantic confusability. It therefore strengthens the case for relational structure, but stops short of determining how such structure should be exploited for runtime disambiguation.

Hierarchical retrieval deserves separate attention because it is the clearest structure-aware response to scaling so far. Both *When Single-Agent with Skills Replace Multi-Agent Systems and When They Fail* and AgentSkillOS suggest that coarse-to-fine routing can mitigate the collapse of flat selection under scale [12, 15]. The core intuition is straightforward: forcing the agent to choose directly from a large flat pool is harder than guiding it through a sequence of smaller local decisions. This can reduce both the search space and the prompt burden. However, hierarchy introduces its own assumptions. It assumes that an early routing decision can be made reliably and that the correct skill is reachable through a small number of functional branches. When skills overlap across multiple categories, or when the hierarchy itself is derived from semantics rather than procedure, these assumptions can fail. Thus, hierarchical skill retrieval appears promising as a scalability aid, but it is not yet clear whether it resolves the deeper procedural disambiguation problem or merely postpones it.

Graph-based and relational organization provide a different response to that problem. By allowing multi-parent and typed relationships, graphs potentially model the reality of skill ecosystems more faithfully than strict trees. SkillNet is the clearest direct skill example, while the knowledge-graph paper on learned skills from demonstrations shows that structured relational

representations are at least plausible at the skill level [16, 14]. The main promise of graph-based organization is that it can encode not only similarity but also composition, dependence, and workflow compatibility. However, graph approaches also face a major difficulty: graph quality depends on how nodes and edges are constructed. If similarity edges are derived from the same shallow semantics that already confuse flat retrieval, then the graph may reproduce the original ambiguity rather than solve it. This means that graph structure is not automatically beneficial; its utility depends on whether the encoded relations capture meaningful procedural distinctions.

2.9 Transferable Ideas from Tool and Document Retrieval

At this point it is reasonable to ask whether ideas from the tool-selection and document-retrieval literature can help. They can, but only as *transferable methodological evidence*, not as direct proof about skill retrieval. For example, SciToolAgent shows that graph-based organization of tools can improve planning and multi-step execution by representing input-output formats, dependencies, and safety relations explicitly [17]. This paper is useful because it demonstrates the value of dependency-aware structured retrieval. However, its unit of organization is the tool, not the skill, and its graph is manually curated in a highly specialized domain. Similarly, LATTICE demonstrates that hierarchical traversal with calibrated path scoring can outperform flat retrieve-then-rerank pipelines for document retrieval [18]. This is methodologically interesting because it shows that LLMs can be used to navigate an index rather than only rerank retrieved candidates. Yet its direct transfer to skills is limited because documents are not procedural capability units. ToolLLM and Gorilla further establish the practical importance of retrieval when capability spaces become large, but they remain fundamentally tool-centric and do not address procedural skill structure [5, 4]. These papers should therefore be used cautiously: they provide ideas for hierarchy, graph structure, reranking, and representation design, but not direct evidence that those methods solve skill retrieval.

2.10 Recent Skill Retrieval Benchmarks and Routers

Several very recent papers make the retrieval problem more concrete. SkillRouter argues that large agent ecosystems make it infeasible to expose all skills in context and studies retrieve-and-rerank selection at approximately 80K candidate skills [19]. Its most important result for this thesis is that hiding the full skill body causes a large routing-accuracy drop, suggesting that names and descriptions often under-specify the information needed for correct selection. This strongly supports the motivation for looking beyond flat metadata. However, SkillRouter primarily emphasizes full-text retrieve-and-rerank performance. It does not isolate which procedural fields inside the full body are responsible for disambiguation.

SkillRet introduces a large-scale skill-retrieval benchmark with public skills, structured semantic tags, a two-level taxonomy, training data, and thousands of evaluation queries [20]. This shows that skill retrieval is becoming a standalone retrieval problem rather than only a downstream

agent-execution issue. Its scale is much larger than the benchmark in this thesis, and therefore provides an important external reference point. At the same time, the present thesis asks a narrower explanatory question: not only whether retrieval can be trained, but what information in a skill representation helps distinguish semantically similar procedures.

Skill Retrieval Augmentation (SRA) formulates a pipeline in which skills are retrieved, incorporated, and applied from large external corpora [21]. This framing is close to the architecture assumed in this thesis, where candidate subsetting happens before the main agent loads or applies full skill artifacts. SRA-Bench also separates retrieval from incorporation and task execution, which reinforces the need to report retrieval metrics and downstream task success separately.

Finally, tool-retrieval work remains methodologically useful even though tools and skills are not identical. ToolRerank shows that hierarchy-aware reranking can improve tool retrieval quality and downstream execution [22], while ToolRet argues that off-the-shelf retrieval models are not automatically tool-savvy [23]. These works support the use of strong reranking baselines and hierarchy-aware comparisons, but the present thesis treats them as transferable retrieval ideas rather than direct evidence about skill artifacts.

2.11 Evaluation Practices and Unresolved Gaps

One of the clearest gaps in the literature emerges when evaluation practices are examined closely. Many papers report overall task success, but fewer isolate retrieval quality itself. SkillsBench measures whether skills help, but not large-library disambiguation [11]. AgentSkillOS and SkillNet demonstrate strong system-level gains, but retrieval, orchestration, and planning improvements are partly entangled [15, 16]. The Claude-skills analysis reveals redundancy and selective-loading pressure, but does not compare retrieval methods experimentally [13]. The Li paper on single-agent skill failure directly targets semantic confusability, which makes it especially important, yet its mitigation strategy is intentionally simple [12]. Across these works, three recurring evaluation limitations appear. First, semantically confusable but procedurally distinct skills are not always tested explicitly. Second, context efficiency is often discussed qualitatively rather than measured directly in tokens, shortlist size, or cost. Third, some structured approaches are compared against relatively weak baselines, making it difficult to isolate how much value comes from structure-aware selection itself rather than broader system engineering improvements.

These limitations matter because the present research problem is more specific than generic retrieval quality. A method may retrieve useful skills on average and still fail on the exact cases that matter most for a personal skill library: cases where surface similarity is high but the correct procedure is different. Likewise, a method may improve overall task performance through better orchestration without demonstrating that the representation itself improves retrieval under confusability. The strongest literature therefore points in a common direction without fully closing the question. Skills are useful. Large skill libraries create real ambiguity. Structure-aware organization appears promising. Yet the literature still does not clearly determine which

representation best preserves procedural distinguishability while remaining efficient and effective at runtime.

2.12 Synthesis and Current Research Questions

The literature therefore suggests that flat skill selection becomes unreliable as skill libraries grow and semantic confusability increases. Although structure-aware approaches are promising, it remains unclear which representational properties most effectively preserve procedural distinguishability while remaining context-efficient and useful in downstream execution. This gap motivates the current working research questions:

RQ1: What information must agent skill representations preserve to distinguish semantically similar but procedurally distinct skills at scale?

RQ2: How do retrieval strategies that use flat text, dense embeddings, structured procedural fields, tree or graph structure, or hybrid reranking differ in accuracy, efficiency, and failure modes when applied to scalable skill libraries?

These questions arise naturally from the literature rather than being imposed on it. Existing work has already shown that flat selection becomes unreliable under scale, that the problem is linked to semantic confusability, and that hierarchical or graph-based organization may help. What remains unresolved is which representational properties most effectively preserve procedural differences between competing skills, and how that effect should be measured. The remainder of the project is therefore best understood not as a generic study of agent scaling, but as a focused investigation into structure-aware skill retrieval under semantic confusability.

2.13 Update Notes for Thesis Revision

Placeholder. Later revision: verify final bibliographic details for the newest 2026 preprints before submission, and decide whether to keep them as preprint evidence or move some into a related-work limitation note.

Chapter 3

Conceptual Framework

3.1 Skill Artifacts, Representations, and Retrieval Policies

This thesis treats skill selection as a layered problem. A *skill artifact* is the complete stored capability unit. In the benchmark used here, the canonical artifact is a skill directory containing a `SKILL.md` file and, where applicable, supporting resources such as scripts, examples, reference files, templates, or external documentation.

A *selection representation* is the compressed or transformed view of a skill used for retrieval. It may be a short card containing a name and description, the full `SKILL.md` artifact, an extracted procedural schema, a dependency/resource card, a hierarchy node, or a graph of typed relations. The representation is not identical to the artifact. A system may store a rich skill artifact while exposing only a short description to the selector.

A *retrieval policy* is the mechanism that uses a representation to produce a candidate set or ranked list. It may be lexical retrieval, dense embedding retrieval, progressive disclosure by the main agent, neural reranking, deterministic field scoring, tree routing, graph retrieval, or a hybrid pipeline. This distinction is essential because a method can fail for two different reasons: the needed procedural information may be absent from the representation, or it may be present but not used correctly by the retrieval policy.

3.2 The Core Failure Mode

The target failure mode is *semantic confusion*: the selected skill is semantically plausible but procedurally wrong. This is different from retrieving a random unrelated skill. The failure is subtle because the wrong skill often shares the same domain, nouns, or surface trigger phrases as the gold skill.

For example, several document skills may all mention PDFs and extraction. One may answer questions from a PDF using page-grounded evidence, another may reconstruct tables from layout,

another may clean OCR output, another may fill form fields, and another may convert formats. A flat description can make these skills look close even though their inputs, workflows, outputs, and success criteria differ.

This thesis defines *procedural distinctness* as the condition where semantically similar skills differ in at least one operationally important axis, such as:

- input type or input state;
- preconditions or assumptions;
- expected output artifact;
- workflow steps and ordering;
- required tools, resources, or external dependencies;
- negative usage boundaries;
- success criteria or verification requirements.

3.3 Representation Conditions

The benchmark exports several representation conditions:

R1 flat metadata. Skill name, family/category, and short description. This approximates the compact view used by progressive disclosure systems.

R2 extracted procedural card. Use conditions, input/precondition fields, output fields, workflow/procedure fields, and boundaries. This tests whether preserving procedural information improves retrieval.

R3 dependency/resource-aware card. R2 plus tools, external dependencies, resources, platform assumptions, and public-source metadata. These fields are useful for feasibility and context loading, but can introduce noise if treated as ordinary positive text.

R4 graph edges. A heterogeneous graph view derived from R2/R3, where skills connect to typed nodes such as triggers, outputs, workflow steps, preconditions, dependencies, resources, and avoid conditions.

Full artifact. The complete `SKILL.md` text. This is important as a strong dense-retrieval baseline because it exposes the most information, but it is costly and can be noisy.

3.4 Retrieval Architecture Families

The architecture families are separated from the representation conditions:

- M0 progressive disclosure.** The main agent sees compact skill cards and decides which full skill documents to load.
- M1 flat lexical retrieval.** BM25 or TF-IDF over R1. This is a lexical control, not the proposed architecture.
- M2 dense retrieval.** Embedding retrieval over R1, R2, or full artifacts.
- M3 structured-card retrieval.** Retrieval over serialized R2 procedural fields.
- M4 tree routing.** Hierarchical selection through family or cluster nodes before selecting a skill.
- M5 graph retrieval.** Retrieval or reranking over R4 relation edges.
- M6 hybrid procedural reranking.** A first-stage retriever produces a candidate set, then a procedural layer reranks candidates.
- M7/M8 provider retrieval and reranking.** Modern API embedding and reranking baselines, currently using Qwen embeddings and Qwen reranking.

The current deterministic schema reranker is treated as **M6-v0**. It is useful as a transparent diagnostic but is not the final proposed structure-aware method. The planned **M6-v1** method should parse the user request into fields and compare those request fields against extracted skill fields. This distinction prevents the thesis from overclaiming that simple text overlap is equivalent to structured reasoning.

3.5 Tree and Graph Structure

Tree and graph methods are structure-aware architectures, not separate information sources. M4 tree routing tests whether a hierarchy can reduce candidate set size without excluding the gold skill too early. M5 graph retrieval tests whether explicit typed relations improve retrieval beyond serialized structured cards.

In the current R4 export, graph edges include:

- `belongs_to_family;`
- `triggered_by;`
- `avoid_when;`
- `has_precondition;`
- `has_workflow_step;`
- `produces_output;`

- `requires_dependency;`
- `has_resource_signal;`
- `uses_resource;`
- `derived_from_public_skill.`

These edges encode the same proposed fields as R2/R3, but they expose them relationally. This allows edge ablations: for example, testing trigger-only retrieval, trigger plus output, trigger plus output plus workflow, and then adding dependencies, resources, and avoid penalties.

The graph should not include benchmark-only edges such as `confusable_with` if those edges are derived from gold labels, closest alternatives, or manual adjudication. Such edges are useful for analysis, but using them for retrieval would leak the benchmark structure. Final M5 retrieval should use only skill-artifact information, public provenance, or unsupervised similarity that would be available outside the evaluation set.

3.6 Working Hypotheses

The experiments are guided by five working hypotheses:

1. Flat metadata will degrade under scale because it under-specifies procedural distinctions.
2. Full skill artifacts will improve over short descriptions in some cases because they expose more information, but they may also introduce noise and cost.
3. Use conditions, output artifacts, and workflow/procedure fields will be the most important positive fields for distinguishing near-neighbour skills.
4. Dependency, resource, and boundary fields should be used conditionally or as penalties rather than appended naively as positive retrieval text.
5. Hybrid retrieval-reranking methods will perform best when dense retrieval provides broad candidate recall and procedural reranking resolves near-neighbour confusion.

Chapter 4

Benchmark Design

4.1 Design Goals

The benchmark is designed to test skill retrieval under the failure mode that motivates the thesis: semantic confusability between procedurally distinct skills. It is not enough for the benchmark to contain many unrelated skills. The benchmark must contain clusters where several skills sound relevant to the same request, but only one skill is procedurally correct.

The benchmark therefore has four goals:

1. create controlled near-neighbour skill clusters;
2. scale the library with generated and public background skills;
3. validate that gold labels remain defensible under non-core competition;
4. test whether proposed representation fields are also visible or extractable in public skill artifacts.

4.2 Skill Library Composition

The active benchmark contains 2401 skills:

- 137 controlled evaluated skills;
- 1800 generated background-scale skills;
- 460 imported public background skills;
- 4 support or email skills.

The controlled evaluated skills are organised into task families such as document and PDF operations, browser automation, GitHub and CI workflows, Hugging Face workflows, observability, office automation, API design, deployment, planning, reading and research, messaging, security, and skill lifecycle operations. These skills form the main controlled benchmark. The public skills are used both as realistic background distractors and as evidence for whether the proposed fields exist or are extractable in real skill artifacts.

4.3 Controlled Confusable Clusters

A valid controlled cluster must satisfy three conditions. First, the skills must overlap in surface meaning, name, description, or trigger phrases. Second, the skills must differ in operationally important procedure. Third, choosing the wrong skill must matter for the resulting workflow or artifact.

Examples include:

- **PDF/document operations:** PDF question answering, layout table extraction, OCR cleaning, form filling, redaction review, and PDF-to-DOCX conversion.
- **CI and GitHub workflows:** CI log root-cause debugging, PR review-comment resolution, repository code review, issue triage, changelog generation, and git safety guardrail installation.
- **Observability:** Prometheus alert writing, dashboard building, distributed trace investigation, SLO breach narrative writing, resilience review, and service-mesh traffic debugging.
- **Skill representation analysis:** skill field auditing, router policy design, skill deduplication, benchmark cluster construction, and graph-edge schema design.

The purpose of these clusters is not to model every possible agent skill. It is to create controlled pressure around the exact failure mode of interest: plausible semantic neighbours with different procedures.

4.4 Background and Public Skills

The 1800 generated background-scale skills provide scale pressure and near-domain distractors. They are designed to be functional enough to compete semantically with evaluated prompts, but they are not the main gold-label targets. Their role is to reveal whether retrieval methods are robust when the library contains many plausible non-core candidates.

The 460 public imported skills improve ecological validity. They introduce authoring styles, broad skills, implicit fields, resources, dependencies, and hierarchical patterns that are less clean than the controlled skills. Public skills are used in three ways:

1. as background scale and distractors;
2. as a field-taxonomy audit corpus;
3. as a separate public-gold validation stratum when a public skill can be manually adjudicated as a stable target.

4.5 Public-Gold Validation Stratum

A separate 32-prompt public-gold stratum was built to test whether externally authored public skills can serve as retrieval targets. This is deliberately separated from the controlled benchmark because public skills are often broader, less atomic, duplicated, or hierarchical.

Manual adjudication found:

- 21 stable strict public-gold cases;
- 7 cases that should be kept only with acceptable alternatives;
- 4 cases that should be revised or excluded before final reporting.

This result is important because it supports a cautious claim. Public skills are valuable for external validity, but they are not clean gold labels by default. Public-gold results should therefore be reported separately as strict public-only, gold-or-acceptable, and cleaned strict subsets.

4.6 Gold Labels and Acceptable Alternatives

Each evaluated prompt has a gold-label intended skill. A gold label is the skill that should be selected if the selector correctly interprets the user’s procedural requirements, not merely the skill whose name or description is most semantically similar to the prompt. A gold label is valid when careful reading shows that the prompt’s requirements align more strongly with the gold skill than with its near neighbours.

Operationally, a skill is treated as the gold label only when it satisfies the following criteria:

1. **Procedural fit:** the skill’s intended use, inputs or preconditions, workflow, output artifact, and success criterion match the request.
2. **Discriminative fit:** at least one concrete requirement in the prompt explains why this skill is better than the listed alternatives.
3. **Near-neighbour pressure:** the alternatives are semantically plausible, but differ from the gold skill on an operational axis such as input state, output type, workflow, dependency, boundary condition, or success criterion.

4. **Atomicity:** the skill is specific enough to be selected as a single procedural capability. Broad router skills or hierarchical skills are not used as strict gold labels unless the prompt explicitly asks for that broad routing behaviour.
5. **No leakage:** the prompt should not reveal the gold skill through exact skill names, copied distinctive phrases, or artificial labels.
6. **Stability under scale:** when public and generated background skills are added, no unrecorded competitor should be clearly more procedurally suitable than the gold skill.

These criteria distinguish the gold label from a merely plausible retrieval hit. For example, in a PDF cluster, a request to extract native text, tables, and metadata from a non-scanned PDF has a different gold skill from a request to OCR a scanned PDF, fill a PDF form, answer questions over a PDF, or convert a PDF to DOCX. All of those alternatives are semantically close because they concern PDFs, but they are procedurally different.

Acceptable alternatives are recorded when a competing skill is genuinely near-equivalent. This avoids treating annotation ambiguity as either a retrieval success or retrieval failure. In public-gold cases, acceptable alternatives are especially important because imported public skills can overlap with controlled skills or with other public skills.

4.7 Validation Rubric

The benchmark is validated through a staged rubric:

1. **Integrity:** all prompts, gold skills, and alternatives resolve.
2. **Procedural alignment:** each prompt contains requirements that justify the gold skill.
3. **Semantic confusability:** each prompt has plausible near-neighbour skills.
4. **Prompt leakage:** prompts should not reveal exact skill names or copied distinctive phrases.
5. **Scale regime:** the library should be large enough to create context and retrieval pressure.
6. **Public field audit:** proposed fields should be observed, extractable, or clearly labelled as proposed normalization fields in public skills.
7. **Public-gold validation:** public skills should be manually atomized and adjudicated before being used as gold targets.
8. **Selector pressure:** baseline methods should show meaningful performance differences rather than all succeeding or all failing randomly.
9. **Downstream validation:** a smaller sample should test whether retrieval quality affects task success.

4.8 Current Validation Status

The active validation status is:

- Step 1 integrity: passed for 137 prompts and 2401 skills.
- Step 2 procedural distinctness: passed for 137/137 prompts and 449/449 gold/alternative pairs.
- Step 2 prompt-specific alignment: passed for 122/137 prompts under MiniLM alignment.
- Step 3 semantic confusability: near-pass, with 116/137 prompts, or 84.7%, having at least two plausible alternatives.
- Step 4 prompt leakage: passed with 0 critical exact-name leaks and 0 high-risk leaks.
- Step 5 scale regime: passed; R1 flat metadata is approximately 123k selector-visible tokens, while full skill artifacts exceed 1M selector-visible tokens.
- Step 6 public field audit: mostly complete; 460/460 public skills audited heuristically and with DeepSeek model-assisted verification.
- Step 6b public-gold validation: manually adjudicated but still needs annotation cleanup.

The main remaining benchmark risk is gold-label stability under realistic public/generated competitors. This risk is handled by manual adjudication, acceptable alternatives, and separate reporting of controlled and public-gold strata.

Chapter 5

Methodology

5.1 Experimental Overview

The methodology is a controlled comparison of representation-and-retrieval conditions. Each method receives the same user prompts and skill library, produces a ranked list of candidate skills, and is evaluated against gold labels and documented acceptable alternatives.

The methodology treats skill selection as a *two-factor design*. The first factor is the representation layer: what information about each skill is visible to the selector. The second factor is the retrieval architecture: how that representation is searched, ranked, or reranked. This separation is essential because a selector can fail either because the representation has lost the relevant procedural distinction, or because the retrieval architecture fails to use information that is present.

The independent variables are therefore:

- the selection representation exposed to the retriever;
- the first-stage retrieval strategy;
- whether a reranker is used;
- whether the reranker uses selection information as structured fields or as ordinary text.

The main dependent measures are top-1 accuracy, top-k recall, mean reciprocal rank (MRR), background or non-core false positives, candidate budget, context cost, latency, and qualitative failure type.

The intended final experiment matrix is shown in Table 5.1. Rows correspond to representation layers and columns correspond to retrieval/reranking architectures. A result is only interpreted as a representation effect when the retriever, reranker, prompt stratum, skill library scale, and candidate budget are held fixed. A result is only interpreted as an architecture effect when the representation layer is held fixed.

Representation layer	Embedding only	Neural reranker	Field-aware reranker	Tree/graph retrieval
R1 flat metadata	Qwen public run; controlled rerun needed. SkillRouter not run.	Qwen public run; controlled rerun needed. SkillRouter not run.	Diagnostic only; weak procedural fields.	Not central.
R2 structured procedural	Qwen public run; controlled rerun needed. SkillRouter not run.	Qwen public run; controlled rerun needed. SkillRouter not run.	Main field-aware target.	Candidate ablation.
R3 dependency/resource-aware	Partial local ablations.	Not complete.	Dependency and boundary ablation.	Candidate graph ablation.
Full SKILL.md artifact	Qwen and SkillRouter run.	Qwen and SkillRouter run.	Qwen or SkillRouter first stage plus local field rerank.	Requires field extraction first.

Table 5.1: Crossed representation-layer and retrieval-architecture design. Blank or partial cells are methodological gaps rather than negative results.

This crossed design also explains why provider results must be interpreted carefully. For example, Qwen reranking over R2 structured cards is not directly comparable with SkillRouter reranking over full SKILL.md artifacts. That comparison changes both the model and the representation. The fair comparison requires running each architecture against the same representation layer, especially R1, R2, and full skill text.

5.2 Representation Conditions

The experiments use the following representation conditions:

R1 flat metadata. Skill name, family, and short description. This approximates the compact representation that a main agent might see in a progressive-disclosure setup.

Full artifact text. The complete SKILL.md text. This exposes the richest information but is expensive and can dilute the signal with examples, resources, and broad documentation.

R2 structured procedural card. Extracted fields for use conditions, preconditions, workflow, outputs, and boundaries.

R3 dependency/resource-aware card. R2 plus external dependencies, resource signals, public-source metadata, and bundled resource files.

R4 graph edges. A typed edge view derived from R2/R3 fields. It is prepared for M5 graph retrieval and graph-aware reranking.

The thesis does not assume that public skill authors always write clean fields. The representation layer is allowed to extract or normalize fields from prose. This is important because the public skill audit shows that many selection signals are present but not always explicitly labelled.

The current fixed selection-information groups are task/use condition, input/precondition, output/artifact, workflow/procedure, dependency/resource/platform, boundary/not-for, and hierarchy/task role. These are not all simple positive retrieval text. Task, output, and workflow are primary disambiguation fields; dependency, boundary, and hierarchy are conditional feasibility or guardrail fields.

5.3 Retrieval and Reranking Methods

The method families are:

M0 progressive disclosure. The main agent sees compact skill cards and decides which full skill documents to load. This is the normal-agent baseline, but it becomes expensive at large scale.

M1 flat lexical retrieval. BM25 or TF-IDF over R1. This is a control for lexical cueing rather than a proposed final architecture.

M2 dense retrieval. Embedding retrieval over R1, R2, or full artifacts. Current provider experiments use Qwen `text-embedding-v4` and the released SkillRouter embedding model; MiniLM is retained mainly as a local construction and validation model.

M3 structured-card retrieval. Retrieval over serialized R2 procedural fields. This tests whether procedural information helps when exposed in a structured card.

M4 tree routing. A planned hierarchy-based architecture: route from root to family or cluster, then to skill. This tests scale efficiency and branch-exclusion failure.

M5 graph retrieval. A planned graph-based architecture over R4 edges. This tests whether explicit relations among triggers, outputs, workflow steps, dependencies, resources, and boundaries help beyond serialized text.

M6-v0 diagnostic schema reranking. The current deterministic local schema reranker. It uses weighted lexical overlap over extracted fields. It is transparent and useful for diagnosis, but too crude to be the final structure-aware method.

M6-v1 field-aware rerank. The current first prototype of the proposed structure-aware method. It parses the request into fields and compares those fields against extracted skill fields.

M7/M8 provider retrieval and reranking. Provider or hosted-model embedding retrieval and neural reranking. Current implementations include Qwen embeddings with Qwen reranking, and SkillRouter embedding with SkillRouter reranking on Hugging Face Jobs.

5.4 M6-v0 Diagnostic Reranker

The existing schema reranker scores lexical overlap between the user request and skill fields. It assigns weight to skill name, description, use conditions, output shape, preconditions, workflow, dependencies/resources, and a negative boundary penalty.

This method is intentionally simple. It tests whether extracted procedural fields contain useful information, but it has limitations:

- it does not parse the user request into structured requirements;
- it uses lexical overlap rather than semantic field matching;
- it can overweight tool or resource words;
- it does not reliably detect broad parent/router skills;
- it treats extracted public-skill fields as clean even when extraction is noisy.

Therefore, M6-v0 should be reported as diagnostic evidence, not as the final proposed retrieval architecture.

5.5 M6-v1 Field-Aware Procedural Reranking

The M6-v1 method better matches the thesis claim. Its pipeline is:

1. A first-stage retriever returns a top-50 or top-100 candidate set.
2. A request parser extracts request-side fields: intended action, input or precondition, desired output, workflow requirement, tool/platform requirement, explicit exclusion, and success criterion.
3. The reranker compares request fields to R2/R3 skill fields.
4. The final score combines first-stage retrieval score with field matches and penalties.

A transparent starting score is:

```
score =  
    0.30 * normalized_first_stage_score  
+ 0.20 * use_when_match  
+ 0.20 * output_match  
+ 0.15 * workflow_match  
+ 0.10 * input_precondition_match
```



```
+ 0.05 * success_criterion_match
+ conditional_dependency_bonus
- boundary_mismatch_penalty
- hierarchy_router_penalty
```

The purpose is not to hand-tune the benchmark until this score wins. The purpose is to test whether extracted selection information improves selection when it is used as field-to-field evidence rather than appended as plain text.

The implemented prototype should be interpreted as M6-v1-local, a lexical field-aware prototype. It is stronger than M6-v0 because it includes request-side field extraction and candidate-recall decomposition. However, it is not yet a robust semantic field matcher. The request parser uses cue-based extraction, and the field matcher mainly uses token overlap rather than learned semantic entailment or contradiction detection. A later version could replace the matcher with a stronger neural or LLM-assisted field matcher while keeping the same information taxonomy.

This distinction matters for interpretation. If M6-v1-local improves a controlled result, the evidence supports the value of the extracted fields and field-to-field matching under a transparent scoring rule. It does not yet prove that the scoring rule is the best possible structure-aware architecture. Conversely, if M6-v1-local fails on public-authored skills, that failure may come from noisy field extraction, weak request parsing, lexical mismatch, public-skill breadth, or duplicated public skills. These sources must be separated before making a general claim about structure-aware retrieval.

The current M6-v1-local analysis therefore records candidate recall separately from final top-1. A first-stage failure means the gold skill never entered the reranker candidate set. A reranking failure means the gold skill was present but not placed first. This separation is important because dense retrievers may be good high-recall candidate generators even when final ordering remains poor.

5.6 SkillRouter Hybrid Experiment

To compare the proposed field-aware reranker against a strong skill-specific neural baseline, the thesis also evaluates a hybrid pipeline:

1. SkillRouter’s released embedding model, `pipizhao/SkillRouter-Embedding-0.6B`, embeds full skill artifacts and the user request.
2. The embedding model produces a first-stage ranked candidate list.
3. Either SkillRouter’s released reranker, `pipizhao/SkillRouter-Reranker-0.6B`, reranks top-20 candidates, or the M6-v1-local field-aware reranker reranks top-20 or top-100 candidates.

This experiment was executed inside a Hugging Face Jobs `t4-small` GPU container. The benchmark snapshot, including skills, prompts, representation files, and scripts, was uploaded to a private Hugging Face dataset. The container downloaded the snapshot, loaded the SkillRouter model weights from Hugging Face, computed first-stage embeddings, and then ran either the SkillRouter neural reranker or the local M6-v1 field-aware reranker inside the same remote environment. Embeddings were not transferred back to the laptop for local reranking.

This design gives a comparison between two second-stage selection policies over the same strong first-stage retriever: learned SkillRouter reranking versus explicit procedural field matching. Candidate budget must be held fixed for a direct reranker comparison. The SkillRouter neural reranker has currently been run at a top-20 budget, while M6-v1-local has been run at both top-20 and top-100 budgets. Therefore, top-20 results are the budget-fair comparison; top-100 M6-v1-local results are reported as a larger-candidate-budget condition unless SkillRouter reranking is also evaluated at top-100.

5.7 M4 Tree Routing

M4 represents skills in a hierarchy:

```
root -> family -> optional cluster -> skill
```

The method first routes the request to one or more families, then ranks only skills inside the chosen branch or branches. It should report:

- family top-1 accuracy;
- family top-n recall;
- final skill top-1/top-5/MRR;
- gold-excluded-by-branch rate;
- candidate reduction ratio;
- context and latency cost.

M4 mainly tests scalability and early-routing risk. It is not the central method for discovering which procedural fields matter.

5.8 M5 Graph Retrieval

M5 uses a heterogeneous graph over skills and information nodes. The current R4 export includes typed edges such as:

- belongs_to_family;
- triggered_by;
- avoid_when;
- has_precondition;
- has_workflow_step;
- produces_output;
- requires_dependency;
- has_resource_signal;
- uses_resource;
- derived_from_public_skill.

Two graph variants are planned. M5a is graph-only retrieval: the request is parsed into requirement nodes and matched to adjacent skill nodes. M5b is hybrid dense-plus-graph reranking: a dense retriever first returns a candidate set, and graph scoring reranks only that subgraph.

Not all edge types should contribute to ranking. For example, `derived_from_public_skill` is provenance evidence for audit and reporting, not retrieval evidence. Similarly, benchmark-derived relations such as manually listed alternatives should not be used by M5, because they would leak evaluation structure into the selector.

Graph edge ablations should test:

- family only;
- trigger only;
- trigger plus output;
- trigger plus output plus workflow;
- adding input/precondition;
- adding dependency/resource;
- adding avoid penalties;
- full graph.

This allows the thesis to distinguish an information claim from an architecture claim. If M5 improves over M6-v1, explicit relation structure may be useful. If M5 does not improve, the thesis can still argue that the fields matter while graph structure may be unnecessary overhead.

5.9 Evaluation Metrics

The retrieval metrics are:

- **Strict top-1 accuracy:** whether the top-ranked skill is the gold skill.
- **Gold-or-acceptable top-1 accuracy:** whether the top-ranked skill is either the gold skill or a documented acceptable alternative.
- **Strict top-k recall:** whether the gold skill appears within the top-k candidate set.
- **Gold-or-acceptable top-k recall:** whether the gold skill or a documented acceptable alternative appears within the top-k candidate set.
- **Mean reciprocal rank:** how highly the strict gold skill is ranked.
- **Non-core top-1 rate:** how often a background or public skill outranks the intended controlled skill.
- **Candidate recall:** whether the first-stage retriever includes the gold skill before reranking.

Strict and acceptable scores are reported separately because they answer different questions. Strict scoring tests whether the selector recovers the intended benchmark target. Acceptable scoring tests whether the selector found a near-equivalent skill that may still be reasonable for task success. Public-gold results require extra care: a public skill may be the correct target, so a generic non-core rate is not meaningful for that stratum. Public-gold failures should instead be labelled as wrong controlled skill, wrong public wrapper, broad parent/router, duplicate or acceptable alternative, extraction failure, first-stage exclusion, or reranking failure.

The efficiency metrics are candidate budget, representation length, selector-visible token cost, latency, provider API calls, and cache behaviour.

Qualitative failure analysis records first-stage exclusion, reranker misweighting, boundary or negation failure, broad parent/router selection, public-skill duplication, underspecified request ambiguity, and acceptable-alternative ambiguity.

5.10 Downstream Validation Plan

Downstream validation remains planned rather than completed. A smaller sample of prompts should be selected after offline retrieval behaviour is stable. The main agent will receive selected candidate skills under fixed conditions, generate task artifacts, and be graded for task success, artifact correctness, evidence use, and misuse of wrong skills.

This step is necessary before claiming that retrieval improvements directly improve final agent task performance. Until then, the thesis should frame current results as retrieval and representation evidence rather than full end-to-end agent reliability evidence.

5.11 Reproducibility

Reproducibility is supported by storing prompts, skill artifacts, representation exports, selector scripts, cached provider outputs, validation reports, public-skill audit outputs, and manual adjudication notes in the project repository. Provider-based experiments should report model names, candidate budgets, cache status, and approximate cost or latency where available.

Chapter 6

Results

6.1 Benchmark Validation Results

The active benchmark contains 2401 skills and 137 controlled evaluated prompts. Table 6.1 summarises the current validation state.

Check	Current result
Integrity	PASS: 137 prompts and 2401 skills resolve.
Procedural distinctness	PASS: 137/137 prompts and 449/449 gold/alternative pairs pass field-audit criteria.
Prompt-specific alignment	PASS with caveat: 122/137 prompts pass MiniLM alignment.
Semantic confusability	WARN/near-pass: 116/137 prompts, or 84.7%, have at least two plausible alternatives.
Prompt leakage	PASS: 0 critical exact-name leaks and 0 high-risk leaks.
Scale regime	PASS: R1 flat metadata is approximately 123k visible tokens; full skill artifacts exceed 1M visible tokens.
Public field audit	Mostly complete: 460/460 public skills audited heuristically and with DeepSeek model-assisted verification.
Public-gold validation	Manually adjudicated: 21 strict keep, 7 keep with acceptable alternatives, 4 revise/exclude.

Table 6.1: Current benchmark validation status.

These results indicate that the benchmark is valid enough for retrieval-method comparison, but not final enough for unrestricted claims. The main remaining risks are public-gold cleanup, non-core winner adjudication, and downstream validation.

6.2 Local Selector Results

Table 6.2 summarises the current deterministic local selectors on the 2401-skill library.

The local results show that structured representations and schema reranking reduce some scale-

Method	Top-1	Top-5	MRR	Non-core top-1
M1 BM25 flat	67.9%	88.3%	0.771	10.9%
M1 TF-IDF flat	60.6%	83.9%	0.713	19.0%
M3 TF-IDF schema	71.5%	94.9%	0.823	7.3%
M6-v0 BM25 to schema rerank	72.3%	93.4%	0.817	3.6%
M6-v0 TF-IDF to schema rerank	73.0%	89.8%	0.808	8.0%

Table 6.2: Local selector results on the active 2401-skill benchmark.

related errors. Compared with flat TF-IDF, TF-IDF schema retrieval improves top-1 from 60.6% to 71.5% and reduces non-core top-1 from 19.0% to 7.3%. However, these are lexical/schema baselines, not a final proof that structure-aware retrieval is solved.

6.3 Field Ablation Results

Field ablations test which representation fields contribute to retrieval performance. Table 6.3 shows selected full-scale cumulative ablations.

Field set	Scorer	Top-1	Top-5	Visible tokens
Description only	TF-IDF	61.3%	83.9%	98,433
Description + use conditions	TF-IDF	70.1%	91.2%	272,829
+ input/preconditions	TF-IDF	67.2%	89.0%	468,875
+ output artifacts	TF-IDF	69.3%	91.2%	507,087
+ workflow/procedure	TF-IDF	73.7%	94.2%	665,434
+ constraints/not-for	TF-IDF	73.7%	94.2%	706,385
+ dependencies/resources	TF-IDF	73.7%	94.9%	885,767
Description only	BM25	66.4%	91.2%	98,433
Description + use conditions	BM25	79.6%	94.9%	272,829
+ output artifacts	BM25	81.0%	97.8%	507,087
+ workflow/procedure	BM25	80.3%	98.5%	665,434

Table 6.3: Selected cumulative field ablations on the full 2401-skill condition.

The ablations support the current field taxonomy. Use conditions provide a large early gain. Output artifacts and workflow/procedure fields provide further procedural discrimination. Constraint, dependency, and resource fields do not behave like simple positive text: they may add cost and noise unless handled conditionally.

6.4 Representation-by-Architecture Coverage

The main experimental object is the interaction between representation layer and retrieval architecture. Table 6.4 summarises which cells have been evaluated on the current or near-current scale. This table is important because some earlier comparisons mixed representation effects with model effects. For example, Qwen reranking over R2 structured cards should not

be compared directly with SkillRouter reranking over full skill artifacts as if only the model changed.

Representation	Qwen embedding	Qwen rerank	SkillRouter embedding	SkillRouter rerank
R1 flat metadata	Public 82 run; controlled rerun needed.	Public 82 run; controlled rerun needed.	Not run.	Not run.
R2 structured procedural	Public 82 run; controlled rerun needed.	Public 82 run; controlled rerun needed.	Not run.	Not run.
Full SKILL.md	Controlled 137 and public 82 run.	Controlled 137 and public 82 run.	Controlled 137 and public 82 hosted run.	Controlled 137 and public 82 hosted run.

Table 6.4: Coverage of the crossed representation-layer and retrieval-architecture comparison.

The current result set is therefore incomplete for the strongest representation-level claim. The active controlled benchmark has strong full-skill comparisons, but it does not yet contain current-scale R1/R2 runs for SkillRouter, nor current-scale controlled R1/R2 reruns for Qwen. The public-gold benchmark has a cleaner Qwen representation comparison across R1, R2, and full artifacts, but it lacks SkillRouter R1/R2 runs. The next experiment cycle should fill these cells before making final claims about representation superiority.

6.5 Public-Gold Horizontal Representation Results

The cleaned public-gold stratum currently provides the clearest horizontal comparison for Qwen because R1, R2, and full skill artifacts were tested on the same 82 prompts and 2401-skill library. Table 6.5 shows that full skill artifacts are not automatically the best retrieval representation.

Representation and architecture	Strict top-1	Accept top-1	Strict top-5	MRR
Qwen R1 embedding	62.2%	72.0%	84.2%	0.720
Qwen R1 + Qwen rerank, top-20	74.4%	85.4%	96.3%	0.837
Qwen R2 embedding	59.8%	73.2%	82.9%	0.714
Qwen R2 + Qwen rerank, top-20	76.8%	90.2%	100.0%	0.861
Qwen full artifact embedding	31.7%	50.0%	58.5%	0.448
Qwen full artifact + Qwen rerank, top-20	68.3%	79.3%	80.5%	0.739

Table 6.5: Public-gold horizontal comparison across representation layers using Qwen embedding and reranking.

The public-gold horizontal comparison suggests that representation compression can help rather than merely remove information. Full SKILL.md artifacts contain more raw content, but generic embedding over full artifacts performs poorly because the signal is diluted by broad instructions,

examples, source-specific wording, and wrapper behaviour. R1 and R2 compact representations perform better for Qwen on public-gold. R2 does not beat R1 by embedding alone, but R2 plus neural reranking is strongest, suggesting that structured procedural information is useful when the architecture can compare it at reranking time.

6.6 Provider and SkillRouter Results

Provider experiments test whether the benchmark remains difficult under stronger modern retrieval models. Table 6.6 shows current 2401-scale Qwen full-skill results.

Method	Top-1	Top-5	MRR	Non-main top-1
Qwen full-skill embedding	52.5%	73.7%	0.617	26.3%
Qwen full-skill + Qwen generic rerank, top-20	62.0%	75.2%	0.684	24.8%
Qwen full-skill + M6-v0 local schema rerank, top-100	77.4%	91.2%	0.840	8.0%
Qwen full-skill + M6-v1-local task/output/workflow rerank, top-100	78.8%	92.0%	0.851	6.6%

Table 6.6: Current Qwen provider results on the active 2401-skill benchmark.

The result suggests that dense retrieval is useful for broad candidate generation but insufficient by itself. Generic Qwen reranking improves over full-skill embedding alone, but it does not close the gap on the controlled procedurally confusable benchmark. Adding a procedural reranking layer improves top-1 more substantially and reduces non-main false positives.

Table 6.7 reports the SkillRouter hosted baseline. This is important because SkillRouter is a skill-specific retrieve-and-rerank model rather than a generic embedding provider.

Method	Top-1	Top-5	MRR	Non-main top-1
SkillRouter full-skill embedding	73.0%	94.2%	0.827	14.6%
SkillRouter full-skill + SkillRouter rerank, top-20	83.2%	97.8%	0.898	4.4%

Table 6.7: SkillRouter hosted results on the controlled 2401-skill benchmark.

SkillRouter changes the interpretation of the provider baselines. The controlled benchmark is not only hard for weak local methods or generic embeddings: a skill-specific embedding model performs much better than Qwen full-skill embedding, and SkillRouter’s neural reranker becomes a strong comparison point for any proposed structure-aware reranker.

6.7 M6-v1 Field-Aware Reranker Results

M6-v1-local fixes the first-stage candidate generator and varies which information fields the reranker uses. Table 6.8 shows the Qwen full-skill first-stage condition.

This result is important because it does not say that adding more structure always helps. The

Field set	Top-1	Top-5	MRR	Candidate R@100	Non-main top-1
Task only	75.2%	88.3%	0.816	93.4%	8.0%
Task + output + workflow	78.8%	92.0%	0.851	93.4%	6.6%
Core: task + input + output + workflow	77.4%	92.0%	0.843	93.4%	6.6%
Core + boundary	77.4%	92.7%	0.841	93.4%	8.0%
All fields	73.0%	89.0%	0.809	93.4%	11.7%

Table 6.8: M6-v1-local field-set comparison with Qwen full-skill candidate generation.

strongest field set is task/use condition plus output and workflow. Adding all fields hurts top-1 and increases non-main false positives, which supports the claim that dependency, boundary, and hierarchy signals must be used conditionally rather than treated as ordinary positive text.

6.8 SkillRouter plus M6-v1 Hybrid Results

The strongest current controlled result comes from combining SkillRouter first-stage retrieval with the M6-v1-local field-aware reranker. Table 6.9 compares the best field-aware hybrid results against the SkillRouter-only baseline.

Method	Budget	Top-1	Top-5	MRR	Non-main
SkillRouter embedding only	–	73.0%	94.2%	0.827	14.6%
SkillRouter + SkillRouter rerank	20	83.2%	97.8%	0.898	4.4%
SkillRouter + M6-v1 task/output/workflow	20	83.2%	95.6%	0.895	5.1%
SkillRouter + M6-v1 task/output/workflow	100	86.9%	98.5%	0.927	0.7%

Table 6.9: Controlled SkillRouter and SkillRouter–M6-v1 hybrid comparison. Top-20 rows are the budget-fair reranker comparison; the top-100 M6-v1 row is a larger-candidate-budget condition.

This result suggests that SkillRouter provides an excellent first-stage candidate set, while explicit procedural fields can improve final ordering when reranking a larger candidate pool. The fair top-20 comparison is cautious: M6-v1 task/output/workflow matches SkillRouter reranking on top-1, but has slightly lower top-5 and MRR. The top-100 M6-v1 condition improves to 86.9% top-1, 98.5% top-5, and 0.927 MRR, but this should be interpreted as a larger-candidate-budget result rather than a direct reranker-only win over SkillRouter’s top-20 reranker. Candidate recall is effectively saturated: SkillRouter includes the gold skill in the top-20 for 97.8% of controlled prompts and in the top-100 for 100.0%. Therefore, remaining controlled failures are mainly reranker-ordering errors rather than candidate-generation errors.

6.9 Public-Gold Results

The public-gold stratum behaves differently from the controlled benchmark. Table 6.10 summarises results before applying final public-gold cleanup.

The public-gold results are a useful caution. On this stratum, the current schema method

Method	Top-1	Top-5	MRR
M1 BM25 flat	62.5%	87.5%	0.715
M1 TF-IDF flat	59.4%	87.5%	0.701
M2a MiniLM description	65.6%	87.5%	0.738
M2b MiniLM full skill	31.2%	71.9%	0.483
M3 TF-IDF schema	50.0%	71.9%	0.607
M6-v0 local schema rerank	34.4–46.9%	75.0–87.5%	0.521–0.631
Qwen R1/R2 embedding	59.4%	87.5%	0.716–0.732
Qwen full-skill embedding	28.1%	65.6%	0.469
Qwen full-skill + Qwen rerank	68.8%	84.4%	0.763
Qwen full-skill + M6-v1-local, best strict top-1	53.1%	78.1%	0.647
SkillRouter full-skill embedding	65.6%	96.9%	0.770
SkillRouter + SkillRouter rerank	65.6%	93.8%	0.788
SkillRouter + M6-v1-local, best strict top-1	62.5%	93.8–96.9%	0.765

Table 6.10: Public-gold selector results before final cleanup.

underperforms flat or description retrieval, while Qwen generic reranking is strongest on strict top-1. SkillRouter embedding performs much better than Qwen full-skill embedding and has very strong top-5 recall, but neither SkillRouter reranking nor M6-v1 field-aware reranking improves strict public-gold top-1. M6-v1-local has high candidate recall on public-gold, approximately 96.9% at top-20 and top-100, but weak top-1 reranking. This does not invalidate the representation claim; it shows that extraction quality, broad public wrappers, duplicates, hierarchy, and semantic field matching must be handled explicitly. Public-gold results should therefore be reported separately from controlled results, and final reporting should distinguish strict gold from acceptable alternatives after the manually marked revise/exclude cases are cleaned.

The generic non-main top-1 metric is not used as a headline public-gold metric because public gold skills are themselves public/background skills. Public-gold failures should instead be classified by stratum and cause: wrong controlled equivalent, wrong public wrapper, broad parent/router skill, duplicate or acceptable alternative, extraction failure, first-stage exclusion, or reranker ordering failure.

6.10 M0 Progressive Disclosure

The existing M0 progressive-disclosure trace covers an older core condition. The current manifest summary reports 129 runs over 43 prompts, with 68.2% top-1 accuracy, 69.8% any-hit rate, 21.7% no explicit skill loaded, and a mean of 0.83 full skill documents loaded.

This baseline is useful because it represents normal main-agent selection behaviour. However, it has not yet been rerun on the active 137-prompt, 2401-skill condition. At full scale, M0 is mainly a context/cost stress test because compact R1 metadata is already approximately 123k visible tokens and full artifacts exceed 1M visible tokens.

6.11 Low-Information and Implicit-Field Cases

Low-information and implicit-field cases test whether methods depend on explicit procedural cues. Performance drops sharply when prompts are underspecified. This suggests that procedural reranking is not simply guessing hidden intent; it needs request evidence. In a real agent system, very underspecified requests may require clarification before retrieval.

The 10 implicit-field stress prompts are now integrated into the main benchmark. Their raw skills remain prose-only, while R2/R3 use conservative extraction to recover procedural fields. This better matches the thesis claim: structure-aware representation should include extraction and normalization rather than assuming perfect author-provided schema fields.

6.12 Interim Interpretation

The current results support four interim conclusions:

1. Scale creates real retrieval pressure, including non-core false positives.
2. Use conditions, output artifacts, and workflow/procedure fields are the strongest current positive fields.
3. Dependency, resource, and boundary fields require field-aware handling rather than naive concatenation.
4. SkillRouter plus M6-v1-local is the strongest controlled result so far, but the same field-aware matcher remains too brittle for public-gold cases.

Downstream task success, M4 tree routing, M5 graph retrieval, and stronger semantic M6-v1 variants remain future work at this stage.

Chapter 7

Discussion

7.1 What the Results Suggest about Representation

The current results support the central thesis framing: skill retrieval is not only an architecture problem, but an information-preservation problem. Flat metadata can retrieve broad topical matches, but it often under-specifies the procedural distinctions that matter when skills are near neighbours. Full skill artifacts expose more information, but dense embedding over long artifacts can be noisy, especially for public-authored skills that include broad wrappers, resources, examples, or hierarchy.

Structured procedural fields improve controlled benchmark performance because they preserve information that flat descriptions compress away. The field ablation results suggest that use conditions, output artifacts, and workflow/procedure fields are currently the strongest positive signals. These fields directly correspond to the difference between a skill that sounds relevant and a skill that is procedurally suitable.

The SkillRouter experiments sharpen this claim. SkillRouter’s embedding model is a strong skill-specific first-stage retriever, and its reranker is a strong neural baseline. When the same SkillRouter first-stage candidates are reranked by M6-v1-local, the task/output/workflow field set matches SkillRouter reranking on top-1 at the same top-20 budget and improves further when M6-v1-local is allowed a larger top-100 candidate budget. This does not prove that the deterministic matcher is generally superior to neural reranking; rather, it shows that explicit procedural fields can add useful ordering evidence after a strong skill-specific embedding model has already produced a high-recall candidate set.

7.2 Why M6-v0 Is Diagnostic, Not Final

The current local schema reranker, M6-v0, is useful because it is transparent. It shows that procedural fields can improve selection when the correct skill appears in the first-stage candidate set. However, it is not strong enough to serve as the final proposed method. It relies on

lexical overlap, does not parse the request into structured requirements, under-models workflow semantics, and handles dependency/resource terms too crudely.

The first M6-v1-local prototype addresses part of this weakness by parsing request-side fields and comparing them against skill-side fields. It improves the controlled Qwen condition and, when paired with SkillRouter first-stage retrieval, performs competitively with the SkillRouter neural reranker at a top-20 budget while giving the strongest controlled result at a larger top-100 budget. It also reveals that task/use condition, output, and workflow are the strongest current field set. But public-gold results show that this deterministic matcher is still not enough for messy public skills. The next methodological improvement is not another broad architecture list; it is better semantic field matching, budget-fair comparisons, statistical uncertainty, and cleaner public-gold adjudication.

The public-gold results make this limitation clear. On public-authored skills, naive schema extraction and overlap can underperform flat or description-based retrieval. This is not surprising: public skills are less standardized, often broader, sometimes duplicated, and sometimes hierarchical. Therefore, the correct conclusion is not that structured representation fails. The stronger conclusion is that structured representation must be paired with reliable extraction and field-aware matching.

7.3 What the SkillRouter Hybrid Adds

The SkillRouter hybrid experiment separates two questions that were previously entangled. First, can a strong skill-specific embedding model retrieve the right candidate? On the controlled benchmark, the answer is mostly yes: candidate recall reaches 97.8% at top-20 and 100.0% at top-100. Second, can explicit procedural fields improve the final ordering of those candidates? On the controlled benchmark, the answer is promising but must be read with budget discipline. At the same top-20 budget, M6-v1-local matches SkillRouter reranking on top-1 but has slightly lower top-5 and MRR. At a larger top-100 budget, M6-v1-local reaches 86.9% top-1 and reduces non-main top-1 to 0.7%. This larger-budget result is evidence that procedural fields can be useful when the candidate set has high recall, not a direct proof that M6-v1-local is a better reranker than SkillRouter under equal compute.

The public-gold stratum gives the opposite lesson. SkillRouter retrieves public-gold candidates well, but M6-v1-local does not improve top-1. This suggests that the public-gold problem is not mainly first-stage recall. It is a field extraction and semantic matching problem: public skills may contain the relevant information, but not in the clean form expected by the current deterministic field matcher.

7.4 Information That Appears Important

The current evidence suggests the following hierarchy of retrieval information:

Strong positive fields. Use conditions, expected output artifact, and workflow/procedure. These most directly distinguish semantically similar skills.

Moderate fields. Input state and preconditions. These are important when the request specifies the available artifact, environment, or starting state.

Conditional fields. Dependencies, resources, tools, platforms, and external services. These help when feasibility matters, but can attract false positives if treated as ordinary positive text.

Penalty fields. Negative boundaries and avoid conditions. These should be used as filters or penalties rather than appended to retrieval text.

Audit/downstream fields. Examples, tests, safety notes, and provenance. These may support validation and execution, but are not always central to first-stage retrieval.

This field hierarchy is still provisional. The thesis should avoid claiming that every public skill already contains these fields explicitly. The more defensible claim is that skill artifacts often contain these procedural signals, sometimes implicitly, and that a retrieval representation layer should extract and normalize them.

7.5 Tree and Graph Methods

Tree and graph methods help clarify the difference between information and architecture. M4 tree routing tests whether a hierarchy can reduce candidate set size and context cost. Its main risk is early branch exclusion: once the correct family is missed, the correct skill cannot be recovered unless multi-branch routing is used.

M5 graph retrieval tests a different claim. It asks whether explicit relations among skills and information nodes improve over serialized structured cards. A graph can represent that a skill produces a particular output, requires a dependency, follows a workflow step, or should be avoided under a condition. Edge ablations can then show whether output edges, workflow edges, dependency edges, or avoid edges actually help retrieval. If graph retrieval improves over M6-v1, the thesis can claim that relational structure matters. If it does not, the thesis can still claim that the fields matter while graph architecture may be unnecessary overhead.

7.6 Benchmark Validity

The benchmark has several strengths. It contains controlled confusable clusters, a large scale condition, public imported skills, public-skill field audits, and manual public-gold adjudication. It also records acceptable alternatives rather than forcing every near-equivalent skill to count as a strict failure.

The benchmark also has limitations. The controlled skills were authored for the research problem, so they may express procedural fields more cleanly than real skill libraries. Public skills help address this, but public-gold validation shows that external artifacts are messy. Some public skills are broad parent skills rather than atomic procedures. Some overlap with controlled equivalents. Some contain procedural information only implicitly. These issues should be reported as validity findings, not hidden.

There is also a measurement limitation in the field ablations. Cumulative field conditions add more text as they add more fields, so some gains may reflect additional vocabulary rather than field identity alone. The current pattern still suggests that use conditions, outputs, and workflow matter, but final reporting should include length-controlled or field-dropout checks before making strong causal claims about individual fields.

7.7 Failure Modes

The current experiments reveal several recurring failure modes:

- **First-stage exclusion:** the correct skill is not included in the candidate set, so no reranker can recover it.
- **Object-procedure confusion:** retrieval focuses on the task object, such as PDF or GitHub, rather than the required procedure.
- **Boundary failure:** negative usage conditions are ignored or treated as positive text.
- **Dependency attraction:** skills using the same tool, platform, or resource are ranked highly even when their outputs differ.
- **Broad parent selection:** a broad public or hierarchical skill is selected instead of the more specific atomic skill.
- **Acceptable alternative ambiguity:** a competing skill is genuinely close enough to count as a reasonable answer.
- **Underspecification:** the user request lacks enough procedural evidence for stable selection.

7.8 Implications for Agent Skill Systems

The practical implication is that large skill libraries should not rely only on skill names and short descriptions. A scalable system should preserve procedural information, separate candidate generation from final selection, and treat dependencies, resources, and boundaries with field-aware logic. It should also distinguish atomic skills from broad parent skills and should ask clarification questions when the request lacks procedural evidence. A strong embedding model

such as SkillRouter can provide high candidate recall, but final selection may still benefit from explicit procedural matching.

For thesis writing, the safest claim is:

Better skill retrieval depends on preserving and using procedural information. Dense retrieval is useful for broad recall, but semantic similarity alone does not reliably distinguish near-neighbour skills. Structure-aware methods are promising when they use extracted fields deliberately, but naive schema text or crude overlap is not sufficient.

7.9 Limitations

Several parts of the project remain incomplete. M6-v1-local has been implemented and paired with SkillRouter first-stage retrieval, but stronger semantic or LLM-assisted field matching has not. M4 tree routing and M5 graph retrieval are planned but not evaluated. Downstream task validation has not yet been run. Latency and cost analysis are incomplete. Public-gold results still require cleanup after manual adjudication. Final result tables also need confidence intervals or paired tests, because several controlled gains correspond to a small number of prompts.

These limitations do not invalidate the current thesis direction, but they constrain the strength of the final claims. Until downstream validation is complete, the thesis should claim evidence about retrieval quality and representation design, not full end-to-end agent reliability.

Chapter 8

Conclusion

This thesis investigates skill representation for scalable agent skill retrieval. The motivating problem is semantic confusion: as skill libraries grow, many skills can sound relevant while requiring different procedures. In such cases, the correct skill may exist, but the retrieval system may surface a plausible yet procedurally inappropriate alternative.

The thesis reframes skill selection as a layered problem involving skill artifacts, selection representations, and retrieval policies. This framing is important because a skill artifact may contain rich procedural information while the selector sees only a compressed card. Conversely, a strong retriever may still fail if the representation hides the distinctions needed for procedural selection.

The current benchmark contains 2401 skills and 137 controlled prompts, with generated scale skills, imported public skills, and a manually adjudicated public-gold stratum. Current evidence suggests that flat metadata and generic full-skill embedding alone are fragile under scale. SkillRouter provides a strong skill-specific retrieval baseline: its embedding plus reranker reaches 83.2% top-1 on the controlled benchmark. The strongest current controlled result combines SkillRouter first-stage retrieval with M6-v1-local task/output/workflow reranking, reaching 86.9% top-1, 98.5% top-5, and 0.927 MRR. However, public-gold results show that naive schema overlap and the first deterministic M6-v1-local matcher are not enough when skills are externally authored, broad, duplicated, or implicit.

The main interim answer to RQ1 is that skill representations should preserve at least use conditions, expected outputs, workflow/procedure, inputs or preconditions, boundaries, and conditional dependency/resource information. The main interim answer to RQ2 is that dense retrieval is useful for broad candidate generation, especially when using a skill-specific model such as SkillRouter, but final selection benefits from procedural reranking when the procedural information is accurately extracted and used field-to-field.

The next technical step is to improve M6-v1 beyond the deterministic local prototype by adding semantic or LLM-assisted field matching, especially for public-authored skills. If time permits, M4 tree routing and M5 graph retrieval should be evaluated as structure-aware architecture

comparisons. The final thesis should then add cost/latency analysis, cleaned public-gold results, and a small downstream validation study.

The expected contribution is not a universal skill-router benchmark or a provider leaderboard. It is a representation-centred account of why semantically similar skills are difficult to retrieve, which procedural information appears to matter, and how retrieval architectures can use that information to make large agent skill libraries more reliable.

Bibliography

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>
- [2] G. Izacard, E. Grave, and S. Riedel, “Atlas: Few-Shot Learning with Retrieval Augmented Language Models,” 2022.
- [3] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language Models Can Teach Themselves to Use Tools,” 2023.
- [4] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, “Gorilla: Large Language Model Connected with Massive APIs,” 2023.
- [5] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, “ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs,” 2023.
- [6] Y. Wu and Y. Zhang, “Agent Skills from the Perspective of Procedural Memory: A Survey,” <https://yashonwu.github.io/agent-skills-survey/>, 2026, preprint.
- [7] Y. Jiang, D. Li, H. Deng, B. Ma, X. Wang, Q. Wang, and G. Yu, “SoK: Agentic Skills – Beyond Tool Use in LLM Agents,” 2026.
- [8] Anthropic, “Agent Skills – Claude API Docs,” <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>, 2025, accessed: 2026-04-11.
- [9] A. Z. Liu, J. Choi, S. Sohn, Y. Fu, J. Kim, D.-k. Kim, X. Wang, J. Yu, and H. Lee, “SkillAct: Using Skill Abstractions Improves LLM Agents,” <https://openreview.net/forum?id=6LG3cIRrF4>, 2024, iCML 2024 Workshop on LLMs and Cognition.
- [10] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An Open-Ended Embodied Agent with Large Language Models,” 2023.
- [11] X. Li, W. Chen, Y. Liu, S. Zheng, X. Chen, Y. He, Y. Li, B. You, H. Shen, J. Sun, S. Wang, Q. Zeng, D. Wang, X. Zhao, Y. Wang, R. B. Chaim, Z. Di, Y. Gao, J. He *et al.*, “SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks,” 2026.

- [12] X. Li, “When Single-Agent with Skills Replace Multi-Agent Systems and When They Fail,” 2026.
- [13] G. Ling, S. Zhong, and R. Huang, “Agent Skills: A Data-Driven Analysis of Claude Skills for Extending Large Language Model Functionality,” 2026.
- [14] “Learning Agent Skills from Demonstrations and Generating Knowledge Graph,” 2026, publication details to be verified before final submission.
- [15] H. Li, C. Mu, J. Chen, S. Ren, Z. Cui, Y. Zhang, L. Bai, and S. Hu, “Organizing, Orchestrating, and Benchmarking Agent Skills at Ecosystem Scale,” 2026.
- [16] Y. Liang, R. Zhong, H. Xu, C. Jiang, Y. Zhong, R. Fang, J.-C. Gu, S. Deng, Y. Yao, M. Wang, S. Qiao, X. Xu, T. Wu, K. Wang, Y. Liu, Z. Bi, J. Lou, Y. E. Jiang, H. Zhu *et al.*, “SkillNet: Create, Evaluate, and Connect AI Skills,” 2026.
- [17] K. Ding, J. Yu, J. Huang, Y. Yang, Q. Zhang, and H. Chen, “SciToolAgent: A Knowledge-Graph-Driven Scientific Agent for Multi-Tool Integration,” *Nature Computational Science*, vol. 5, pp. 962–972, 2025.
- [18] N. Gupta, W.-C. Chang, N. Bui, C.-J. Hsieh, and I. S. Dhillon, “LLM-Guided Hierarchical Retrieval,” <https://openreview.net/forum?id=p0gxvUoZM>, 2025.
- [19] Y. Zheng, Z. Zhang, C. Ma, Y. Yu, J. Zhu, Y. Wu, T. Xu, B. Dong, H. Zhu, R. Huang, and G. Yu, “SkillRouter: Skill Routing for LLM Agents at Scale,” 2026.
- [20] H. Cho, R. Kang, and Y. Kim, “SkillRet: A Large-Scale Benchmark for Skill Retrieval in LLM Agents,” 2026.
- [21] W. Su, J. Long, Q. Ai, Y. Tang, C. Wang, Y. Tu, and Y. Liu, “Skill Retrieval Augmentation for Agentic AI,” 2026.
- [22] Y. Zheng, P. Li, W. Liu, Y. Liu, J. Luan, and B. Wang, “ToolRerank: Adaptive and Hierarchy-Aware Reranking for Tool Retrieval,” in *Proceedings of LREC-COLING 2024*, 2024, pp. 16 263–16 273.
- [23] Z. Shi, Y. Wang, L. Yan, P. Ren, S. Wang, D. Yin, and Z. Ren, “Retrieval Models Aren’t Tool-Savvy: Benchmarking Tool Retrieval for Large Language Models,” 2025.

Appendix A

Benchmark Rubric

This appendix summarises the benchmark validation loop used in the project. The full operational rubric is maintained in `skill_benchmark/notes/benchmark_methodology_rubric.md`.

A.1 Validation Steps

1. **Skill and prompt integrity.** Every prompt has a unique identifier, every gold skill exists, every listed alternative exists, and skill metadata can be parsed.
2. **Procedural requirement alignment.** Each prompt must contain concrete requirements that justify why the gold skill is better than its alternatives.
3. **Semantic confusability.** Alternatives should be plausible neighbours, not random unrelated distractors.
4. **Gold-label stability and leakage.** Prompts should not reveal the answer through exact skill names, copied phrases, or hidden assumptions.
5. **Scale regime.** The benchmark should include enough skills to create retrieval pressure and context pressure.
6. **Public-skill field audit.** Public skills should be checked for observed, extractable, and proposed representation fields.
7. **Public-gold validation.** Public skills can be used as gold targets only after atomization, leakage checks, semantic alternatives, and manual adjudication.
8. **Offline selector evaluation.** Methods should be compared on top-1, top-k, MRR, non-core false positives, and candidate budget.
9. **Progressive-disclosure baseline.** The normal main-agent skill-selection pattern should be evaluated where feasible.
10. **Downstream validation.** A smaller sample should test whether retrieval differences affect final task artifacts.

A.2 Current Pass/Warning Summary

Step	Current status
Integrity	PASS: 137 prompts and 2401 skills resolve.
Procedural distinctness	PASS: 137/137 prompts and 449/449 pairs pass.
Prompt-specific alignment	PASS with caveat: 122/137 prompts pass MiniLM alignment.
Semantic confusability	WARN/near-pass: 116/137 prompts pass.
Prompt leakage	PASS: 0 critical and 0 high-risk leaks.
Scale	PASS: 2401 skills; R1 approximately 123k visible tokens.
Public field audit	Mostly done: 460/460 public skills audited.
Public-gold	Manually adjudicated; cleanup required.
Downstream validation	Planned but not executed.

Table A.1: Current validation summary.

Appendix B

Prompt and Skill Examples

This appendix gives representative examples of the intended benchmark pressure. The exact prompt files are stored under `skill_benchmark/prompts/` and `skill_benchmark/prompts_public_gold/`.

B.1 PDF Table Extraction

User request: extract a table from a PDF while preserving row and column structure.

Gold skill: PDF layout table extractor.

Plausible alternatives: PDF question answerer, PDF OCR cleaner, document field extractor, PDF-to-DOCX converter.

Procedural distinction: the gold skill must inspect layout and reconstruct table structure. A PDF question-answering skill may ground answers in pages but not reconstruct tables. OCR cleaning handles scanned text uncertainty. Conversion changes format but is not optimized for table reconstruction.

B.2 GitHub CI Root Cause

User request: inspect a failed CI run and identify the likely root cause from logs.

Gold skill: CI log root-cause debugger.

Plausible alternatives: deployment build triager, PR review-comment resolver, repository code reviewer, release changelog generator.

Procedural distinction: the gold skill must parse CI logs and isolate the failing command, error, dependency, or environment condition. Review-comment resolution and code review are semantically adjacent GitHub workflows but require different inputs and outputs.

B.3 Observability Trace Diagnosis

User request: diagnose latency across a distributed trace path.

Gold skill: distributed trace investigator or implicit trace path diagnoser, depending on the prompt stratum.

Plausible alternatives: latency anomaly detector, SLO breach checker, service dependency mapper, metrics overview.

Procedural distinction: trace diagnosis follows spans and service boundaries. A metrics overview summarizes signals, and a dependency mapper describes architecture, but neither necessarily reconstructs a trace path.

B.4 Public-Gold Example: Figma Implementation

User request: implement an existing Figma node as production UI code with close visual parity.

Gold skill: public Figma implement-design skill.

Plausible alternatives: broad Figma parent skill, Figma generate-design skill, generic frontend implementation skill.

Procedural distinction: the gold skill targets an existing Figma node and produces code. A broad parent skill may route to several subskills, while generate-design writes or creates design content rather than implementing an existing node.

B.5 Public-Gold Ambiguity Example

Some public-gold prompts are not stable strict targets. For example, a public OCR skill may be valid for scanned PDF OCR, but a controlled OCR-cleaning skill may be better if the prompt asks for uncertain recognition regions and page anchors. These cases are either revised, excluded, or marked with acceptable alternatives before final reporting.